

ASSESSMENT TOOL 1 – CONSTRAINT-BASED PROBLEM SOLVING

Q1. I/O Communication Mechanism for a Real-Time Embedded System

1.1 Constraint Analysis

- Low CPU utilization (<15%) rules out polling-based I/O, since continuously checking sensor status would consume excessive CPU cycles; an interrupt-driven approach is mandatory.
- A 1 ms interrupt period from multiple sensors means the system must service interrupts frequently but each service routine must be extremely short to stay within the CPU budget.
- Vectored interrupt handling is specified, meaning each sensor's interrupt must map directly to its own dedicated Interrupt Service Routine (ISR) via an interrupt vector table, avoiding the overhead of a single polled handler checking every source.
- The choice between memory-mapped and I/O-mapped I/O directly affects instruction-level latency for accessing sensor registers.

1.2 Solution Design

- Use vectored interrupts: assign each sensor its own interrupt line/vector connected to a Programmable Interrupt Controller (PIC/APIC). On an interrupt, the hardware automatically jumps to the specific ISR address from the vector table — eliminating the need to poll or sequentially test each device, which keeps ISR entry latency minimal.
- Keep each ISR minimal ('top half'): the ISR only acknowledges the interrupt and copies the raw sensor sample into a buffer; any heavier processing (filtering, computation) is deferred to a lower-priority task ('bottom half' / deferred procedure), which keeps CPU time spent in interrupt context very small relative to the 1 ms period.
- Select Memory-Mapped I/O (MMIO) over port-mapped I/O: MMIO allows sensor registers to be accessed using regular load/store instructions (rather than special IN/OUT instructions), which most modern embedded cores execute in a single cycle and which allow the compiler to optimize access alongside normal memory operations — giving lower and more predictable latency.
- Enable interrupt priority levels so higher-priority safety sensors preempt lower-priority ones, ensuring the tightest-deadline sensor is always serviced first.

1.3 Justification

With N sensors each interrupting every 1 ms and an ISR taking only a few microseconds (thanks to the top-half/bottom-half split), total CPU time spent servicing interrupts stays well under the

15% budget. Vectored interrupts remove the search/poll overhead of identifying the interrupting device, and MMIO's compatibility with normal instruction pipelines minimizes per-access latency — together satisfying both the CPU-utilization and low-latency constraints.

Q2. I/O Subsystem for High-Speed Storage

Constraints: High-speed storage requires throughput > 3 GB/s; must use NVMe over PCIe; CPU should not handle bulk data transfer; integrate DMA controller with interrupt notification.

2.1 Constraint Analysis

- A throughput requirement above 3 GB/s immediately excludes older interfaces such as SATA (max ~600 MB/s) and mandates a high-bandwidth serial interface — PCIe-based NVMe, which can deliver several GB/s per lane group (e.g., PCIe 4.0 x4 $\approx$ 7.9 GB/s).
- The requirement that the CPU must not handle bulk data transfer means the processor cannot copy data word-by-word (Programmed I/O); this necessitates DMA (Direct Memory Access) for the actual payload movement.
- Interrupt notification must still be used so the CPU is informed only upon completion of a transfer, not during it — preserving CPU availability for other tasks.

2.2 Solution Design

- **Storage Interface:** Use an NVMe SSD connected over PCIe (Gen4 x4 or higher) as the storage device, since NVMe is purpose-built for low-latency, high-throughput, multi-queue access to flash storage and easily exceeds the 3 GB/s requirement.
- **Command/Queueing Model:** NVMe uses paired Submission and Completion Queues residing in host DRAM; the CPU only writes a command descriptor (pointing to source/destination buffers) into the submission queue — it never touches the actual data payload.
- **DMA Integration:** The NVMe controller's own DMA engine reads/writes data directly between the SSD and system DRAM using the addresses provided in the command descriptor, completely bypassing the CPU for the bulk transfer.
- **Interrupt Notification:** Upon completion, the NVMe controller posts a completion queue entry and raises an MSI-X interrupt to notify the CPU that the transfer is done, allowing the CPU to simply pick up the result rather than actively monitor progress.

2.3 Justification

This design meets throughput by using PCIe/NVMe's high per-lane bandwidth, satisfies the 'no CPU bulk transfer' constraint through the NVMe controller's built-in DMA engine, and keeps CPU

overhead minimal by using MSI-X interrupts only at completion rather than requiring any CPU involvement during the transfer itself — an architecture consistent with how modern NVMe SSDs actually operate.

Q3. Multi-Device Communication Architecture

Constraints: 6 peripherals using USB, 2 high-speed devices using Thunderbolt; avoid interrupt starvation; use hardware and software interrupts; ensure fair bus arbitration.

3.1 Constraint Analysis

- A mixed set of 6 USB peripherals and 2 Thunderbolt devices means the architecture must support two different bus protocols concurrently, each with different bandwidth and topology characteristics.
- Avoiding interrupt starvation requires that no single device (particularly a high-frequency one) can monopolize the CPU by repeatedly interrupting it, starving other devices of service.
- Both hardware interrupts (from physical devices) and software interrupts (for background/internal tasks) must coexist, requiring a priority and scheduling scheme.
- Fair bus arbitration is needed because both USB and Thunderbolt are shared buses where multiple devices contend for the same physical channel bandwidth.

3.2 Solution Design

- **Bus Topology:** Connect the 6 USB peripherals through a USB host controller (e.g., USB 3.2) using a tiered-star topology via a hub, and connect the 2 high-speed devices via Thunderbolt, which itself tunnels PCIe and DisplayPort over a daisy-chainable, higher-bandwidth link.
- **Arbitration:** USB uses host-controller-scheduled, token-based arbitration where the host polls/schedules transactions in frames/microframes, and Thunderbolt uses packet-based arbitration with bandwidth reservation for isolated PCIe tunnels — both ensure no single device can seize the bus indefinitely, since transfers are scheduled in bounded time slots.
- **Interrupt Priority Scheme:** Assign hardware interrupts from Thunderbolt (high-speed, latency-sensitive) a higher priority level than USB peripheral interrupts, but implement a round-robin or weighted fair-queueing scheme among same-priority hardware interrupt sources so that no single USB device's frequent interrupts block others.
- **Preventing Starvation:** Use interrupt coalescing/batching for high-frequency devices (grouping several events into one interrupt) and enforce a maximum service quantum per

device per scheduling round, guaranteeing every device gets bus and CPU time within a bounded interval.

- Software Interrupts: Reserve software interrupts (traps) for lower-priority, non-time-critical internal tasks (e.g., deferred processing, OS housekeeping), scheduled only when no pending hardware interrupt requires immediate service, so they never block latency-sensitive hardware events.

3.3 Justification

Combining scheduled bus arbitration (native to USB and Thunderbolt) with a priority-plus-round-robin interrupt scheme ensures every one of the 8 devices receives guaranteed, bounded access to both the bus and the CPU, directly satisfying the fairness and anti-starvation constraints while still allowing latency-critical Thunderbolt traffic to be serviced promptly.

Q4. Data Acquisition System Design

Constraints: Continuous data stream at 500 MB/s; minimal CPU intervention required; must compare DMA transfer vs interrupt-driven I/O; use memory-mapped I/O for device registers.

4.1 Constraint Analysis

- A sustained 500 MB/s stream is far beyond what interrupt-per-word or interrupt-per-small-packet handling could support without overwhelming the CPU; at typical word sizes this could require millions of interrupts per second under a naive interrupt-driven design.
- 'Minimal CPU intervention' points strongly toward DMA-based bulk transfer rather than CPU-mediated data movement.
- Device registers (control/status/configuration) still need to be accessed with low latency, for which memory-mapped I/O is specified.

4.2 DMA Transfer vs Interrupt-Driven I/O — Comparison

Aspect	Interrupt-Driven I/O	DMA Transfer
CPU Involvement	CPU copies every data unit itself inside the ISR	CPU only sets up transfer; DMA controller moves data independently
Suitability for 500 MB/s	Poor — interrupt overhead per unit would saturate the CPU	Excellent — CPU is free during the bulk transfer
Interrupt Frequency	One interrupt per data unit (very high rate)	One interrupt per completed block/buffer (very low rate)
Latency per Transfer	Higher due to context-switch overhead on every unit	Lower per-byte overhead; efficient for large, continuous streams

Aspect	Interrupt-Driven I/O	DMA Transfer
Best Use Case	Low-frequency, small, sporadic transfers	High-throughput, continuous, large-block transfers

4.3 Solution Design

- Use DMA in scatter-gather / ring-buffer mode: configure the DMA controller to continuously fill a circular buffer in DRAM from the acquisition device without CPU involvement for each sample.
- Use memory-mapped I/O for the device's control/status/configuration registers (start/stop acquisition, buffer pointers, sample rate) so the CPU can configure the device using ordinary load/store instructions with minimal latency.
- Generate an interrupt only when a buffer segment is full (block-level interrupt, e.g., every few milliseconds) rather than per sample, so the CPU is invoked only to hand off the filled buffer for processing/storage — keeping CPU intervention to a minimum while still meeting the 500 MB/s throughput.
- Use double-buffering (ping-pong buffers) so the DMA can continue filling the next buffer while the CPU/software processes the previous one, ensuring the continuous 500 MB/s stream is never interrupted.

4.4 Justification

DMA transfer is clearly superior to interrupt-driven I/O for this workload: at 500 MB/s, servicing each unit with an individual interrupt would consume the CPU entirely, whereas DMA with block-level interrupts and double buffering achieves the required throughput with only a handful of interrupts per second, satisfying the minimal-CPU-intervention constraint while MMIO keeps control-register access fast.

Q5. Interrupt Handling Mechanism Design

Constraints: Support nested interrupts; maximum interrupt latency < 10 μ s; use vectored interrupts for high-priority devices; use software interrupts for background tasks.

5.1 Constraint Analysis

- Nested interrupt support means a higher-priority interrupt must be able to preempt an ISR currently servicing a lower-priority interrupt, requiring a priority-based interrupt controller and re-entrant interrupt handling in the processor.

- A strict maximum latency of 10 μ s from interrupt assertion to ISR execution demands a fast, low-overhead dispatch mechanism and short critical sections where interrupts might otherwise be disabled.
- Vectored interrupts are required for high-priority devices so that the correct ISR is dispatched directly by hardware, without a slow software polling loop to identify the source.
- Software interrupts must be reserved for background tasks, which should never be allowed to block or delay a hardware interrupt's servicing.

5.2 Solution Design

- **Interrupt Controller:** Use a Nested Vectored Interrupt Controller (NVIC-style, as in ARM Cortex-M processors) that supports multiple priority levels and automatically stacks/restores processor context, enabling true hardware-assisted preemption with minimal software overhead.
- **Priority Assignment:** Assign the highest priority levels and dedicated vector table entries to time-critical, high-priority devices, ensuring their ISR address is fetched directly from the vector table without any software dispatch delay.
- **Nesting Policy:** Allow an incoming interrupt to preempt a currently executing lower-priority ISR immediately (tail-chaining/late-arrival support in hardware), while same- or lower-priority interrupts are queued until the current ISR completes — this bounds worst-case latency for the highest-priority device.
- **Latency Budget:** Keep each high-priority ISR extremely short (a few instructions to acknowledge and capture data, deferring further work), and minimize or eliminate long critical sections with interrupts disabled, so that the hardware's vector-fetch-and-dispatch time plus ISR entry stays well within the 10 μ s bound.
- **Software Interrupts for Background Tasks:** Implement background/non-critical work (logging, housekeeping, deferred computation) as software interrupts/traps issued at the lowest priority level, or as tasks scheduled in a 'bottom-half' mechanism that only executes when no pending hardware interrupt requires service — guaranteeing they can never delay a high-priority hardware interrupt.

5.3 Justification

A priority-based nested vectored interrupt controller directly satisfies the nesting requirement by allowing hardware-managed preemption; assigning dedicated vectors to high-priority devices removes dispatch-search overhead, keeping latency low; and confining background work to low-

priority software interrupts ensures it can never interfere with the 10 μ s latency bound required for high-priority hardware events.

— *End of Solutions* —